

END-TO-END AWS DEVOPS IMPLEMENTATION

Setup CI/CD with AWS CodePipeline

Automating Continuous Integration and Continuous Deployment for a Spring Boot Library Management Application using AWS CodeBuild, Elastic Beanstalk, and GitHub.

AWS CodePipeline V2

AWS CodeBuild

AWS Elastic Beanstalk

GitHub Webhooks

Java Corretto 21

Maven 3.x

Docker Privileged

1. What is CI/CD?

Continuous Integration (CI)

CI is the engineering practice of automatically integrating source code modifications from multiple software contributors into a central repository. Automated test suites and compile validation runs verify each commit, guaranteeing that incoming changes preserve application stability.

Continuous Deployment (CD)

CD automates the release of thoroughly tested, validated software packages into target production and staging environments without manual intervention, dramatically reducing release lead time and deployment risk.

2. Why AWS CodePipeline and AWS CodeBuild?

- **AWS CodePipeline:** A fully managed continuous delivery and orchestration service that models, visualizes, and coordinates your software release lifecycle stages for fast, zero-downtime, and reliable updates.
- **AWS CodeBuild:** A fully managed build engine that provisions isolated containerized compute, compiles application source code, executes automated unit and integration tests, and generates distributable runtime artifacts (such as executable Spring Boot JAR files).
- **Native Integration with GitHub:** CodePipeline and CodeBuild seamlessly hook into GitHub repositories via event-driven webhooks, triggering pipeline executions instantly whenever changes land on designated branches.

Prerequisites Checklist: Before initiating pipeline construction, confirm that:

1. An active **AWS Elastic Beanstalk** environment running Amazon Linux 2023 with Corretto 21 is provisioned for your Spring Boot application.
2. Your Spring Boot project repository (SantoshMane07/library-management) is published and accessible on **GitHub**.

3. Creating the Pipeline in AWS CodePipeline

Log in to your AWS Management Console, navigate to the **CodePipeline** service under Developer Tools, and click the orange **Create pipeline** button.

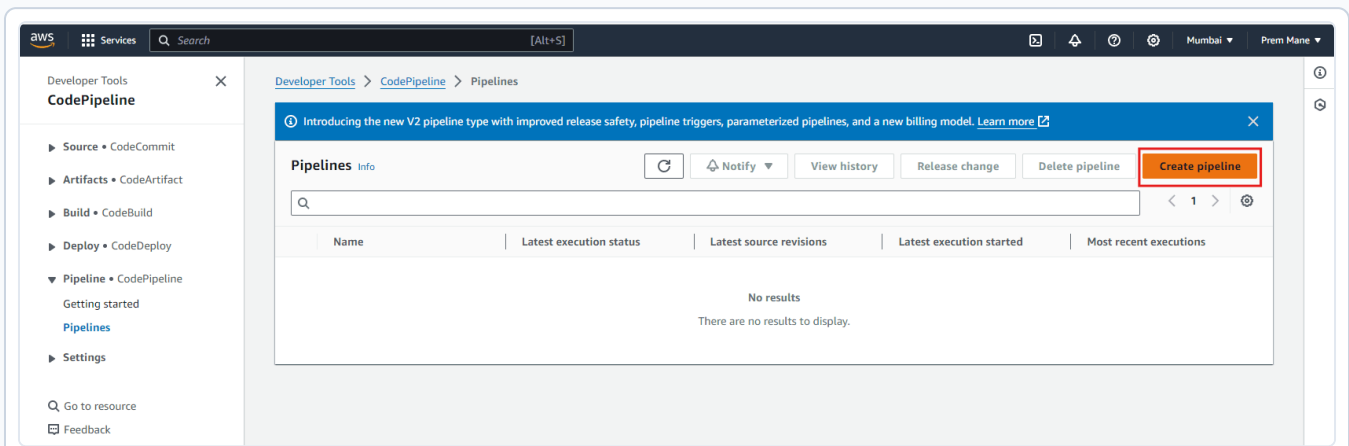


Figure 3.1: CodePipeline Console Dashboard highlighting the "Create pipeline" action

Step 1 Choose Creation Option

On the creation mode selection screen, choose **Build custom pipeline** to configure stages manually from scratch to suit our Spring Boot architecture, then click **Next**.

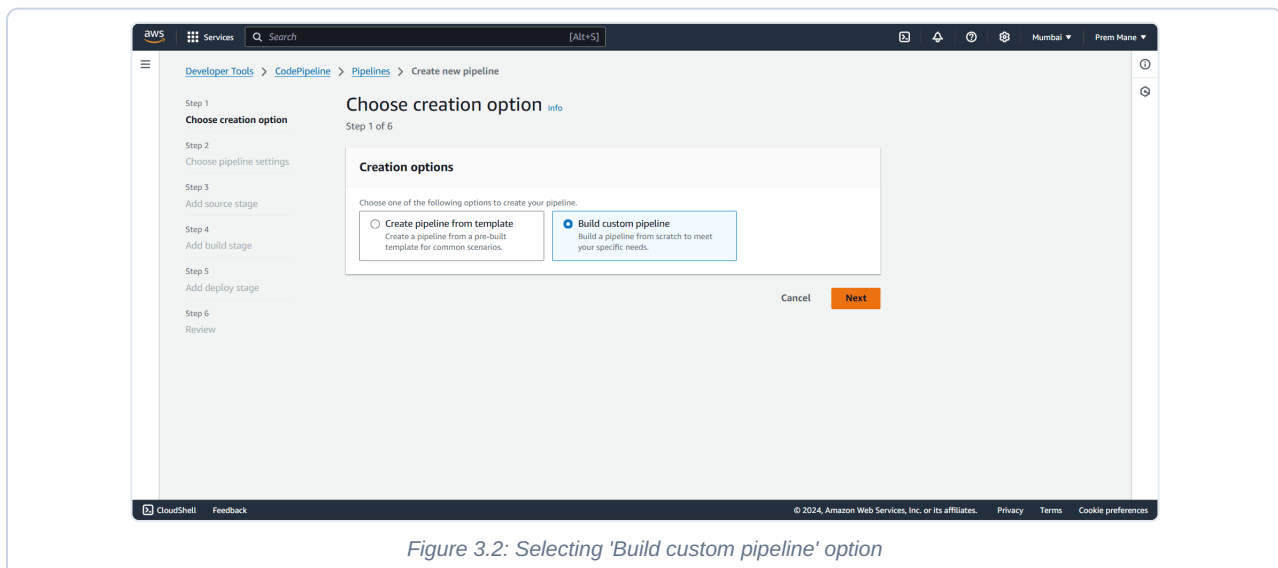


Figure 3.2: Selecting 'Build custom pipeline' option

Step 2 Choose Pipeline Settings

Configure core pipeline parameters, execution concurrency, and Identity and Access Management (IAM) permissions:

- **Pipeline name:** Enter a recognizable identifier: `library-management-service-pipeline`.
- **Pipeline type & Execution mode:** Select **Queued (Pipeline type V2 required)**. In Queued mode, executions are processed sequentially in the exact order they arrive, avoiding race conditions during builds.
- **Service role:** Select **New service role**. CodePipeline automatically generates the role named `AWSCodePipelineServiceRole-ap-south-1-library-service-pipeline` with the necessary cross-service invocation policies.

Choose pipeline settings info
Step 1 of 5

Pipeline settings

Pipeline name
Enter the pipeline name. You cannot edit the pipeline name after it is created.

No more than 100 characters

Pipeline type
You can no longer create V1 pipelines through the console. We recommend you use the V2 pipeline type with improved release safety, pipeline triggers, parameterized pipelines, and a new billing model.

Execution mode
Choose the execution mode for your pipeline. This determines how the pipeline is run.
☐ Superseded
A more recent execution can overtake an older one. This is the default.
Executions are processed one by one in the order that they are queued.
☒ **Queued (Pipeline type V2 required)**
Executions are processed one by one in the order that they are queued.
☐ Parallel (Pipeline type V2 required)
Executions don't wait for other runs to complete before starting or finishing.

Service role
☒ **New service role**
Create a service role in your account.
☐ Existing service role
Choose an existing service role from your account.

Role name

Type your service role name
☒ Allow AWS CodePipeline to create a service role so it can be used with this new pipeline

Figure 3.3: Configuring Pipeline Name, Execution Mode, and IAM Service Role

Expand the **Advanced settings** section below the execution mode. Leave both the **Artifact store** (Default location in Amazon S3) and the **Encryption key** (Default AWS Managed Key) selected as default, then click **Next**.

Variables
You can add variables at the pipeline level. You can choose to assign the value when you start the pipeline. Choosing this option requires pipeline type V2. [Learn more](#)

No variables defined at the pipeline level in this pipeline.

You can add up to 50 variables.

The first pipeline execution will fail if variables have no default values.

Advanced settings

Artifact store
☒ **Default location**
Use the default artifact store (Amazon S3 codigipeline-ap-south-1-26495530253) designated in the same region and account as your pipeline.
☐ Custom location
Choose an existing S3 location from your account in the same region and account as your pipeline.

Encryption key
☒ **Default AWS Managed Key**
Use the AWS managed customer master key for CodePipeline in your account to encrypt the data in the artifact store.
☐ Customer Managed Key
To encrypt the data in the artifact store under an AWS KMS customer managed key, specify the key ID, key ARN, or alias ARN.

Figure 3.4: S3 Artifact Store and Default AWS Managed Key settings

Step 3 Add Source Stage (GitHub Integration)

Connect your GitHub version control repository to provide input code to CodePipeline:

- **Source provider:** Select **GitHub (Version 1)** from the dropdown.
- **GitHub Connection:** Click the **Connect to GitHub** button to grant AWS CodePipeline OAuth access. The status will confirm *Connected*.
- **Repository:** Choose your Spring Boot project repository: `SantoshMane07/library-management`.
- **Branch name:** Select `master`. Pushes to this branch will automatically trigger pipeline executions.

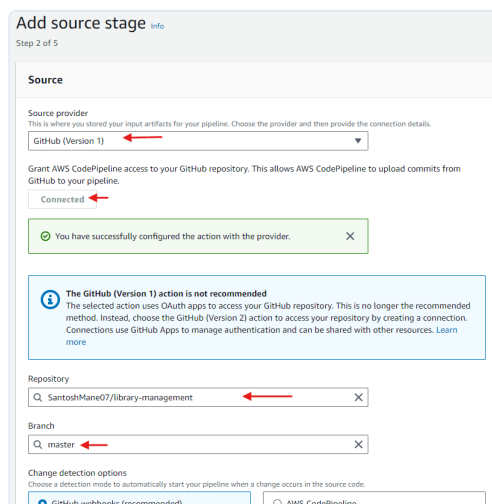


Figure 3.5: Connecting GitHub repository and selecting master branch

Under **Change detection options**, choose **GitHub webhooks (recommended)**. Webhooks provide instant, push-based trigger notifications whenever new commits are merged into the target branch. Click **Next**.

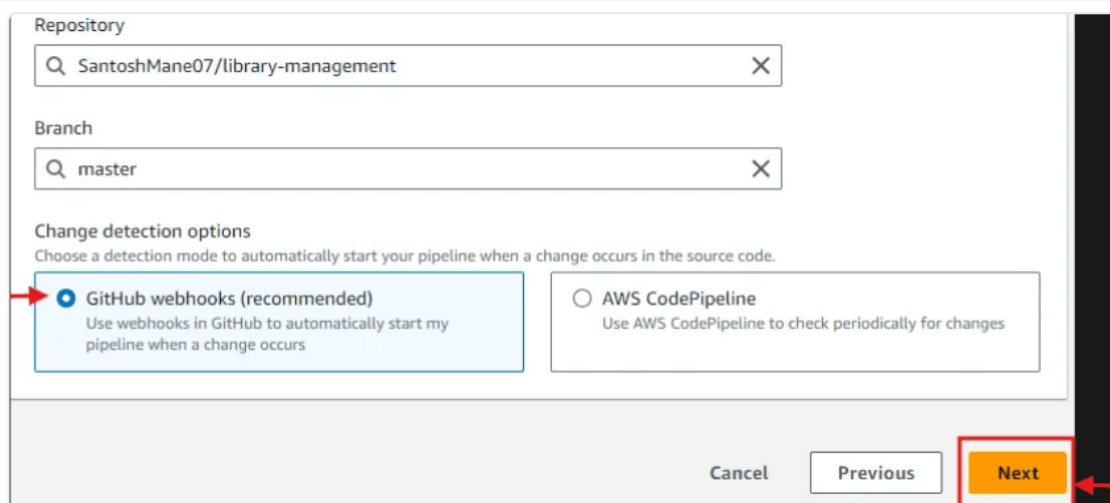


Figure 3.6: Enabling GitHub webhooks for automated change detection

Step 4 Add Build Stage & Create AWS CodeBuild Project

Configure the build engine responsible for running tests and compiling your Spring Boot application:

- **Build provider:** Select **AWS CodeBuild**.
- **Region:** Select **Asia Pacific (Mumbai)** (ap-south-1) or your closest region.
- **Input artifacts:** Select **SourceArtifact**.

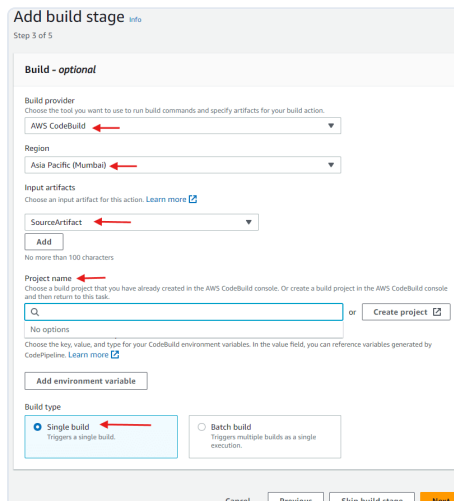


Figure 3.7: Selecting AWS CodeBuild and clicking 'Create project'

Since we do not have an existing CodeBuild project, click the **Create project** button. A modal opens to define the build environment:

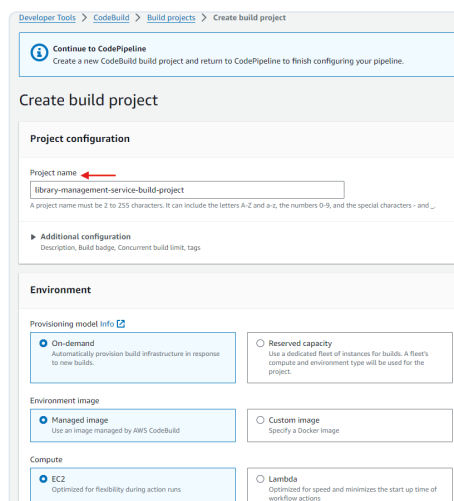


Figure 3.8: Specifying Project Name: library-management-service-build-project

CONFIGURATION FIELD	SELECTED SETTING	PURPOSE / NOTE
Project Name	library-management-service-build-project	Unique build job identifier
Provisioning Model	On-demand	Dynamic container provisioning per build run
Environment Image	Managed image	AWS-provided and maintained base container
Compute	EC2 (3 GB memory, 2 vCPUs)	Sufficient capacity for Maven compilations
Operating System	Amazon Linux	Optimal compatibility with AWS SDK and tools
Runtime(s) & Image	Standard / aws/codebuild/amazonlinux2-x86_64-standard:5.0	Latest LTS base image
Service Role	New service role (codebuild-library-management-service-...)	Grants CodeBuild S3 artifact read/write and CloudWatch logging access
Timeout	1 Hour (Queued: 8 Hours)	Guards against infinite build loops

Figure 3.9: Operating system, runtime, service role, and timeout settings

In the **Additional configuration** section, leave the Privileged flag and VPC settings at default for now (we will enable Docker privileged mode later for containerized tests). Keep the default compute specification (3 GB memory, 2 vCPUs).

Privileged
☐ Enable this flag if you want to build Docker images or want your builds to get elevated privileges

Report auto-discover [Info](#) ☒
☐ Disable report auto-discover

Auto-discover directory - optional

CodeBuild will search for supported report file types in this directory. "*" by default

Certificate
 If you have a self-signed certificate or a certificate signed by a certification authority, choose the option to install it from your S3 bucket.
☒ Do not install any certificate ☐ Install certificate from your S3 bucket

VPC
 Select a VPC that your AWS CodeBuild project will access.

Compute
☒ 3 GB memory, 2 vCPUs
☐ 7 GB memory, 4 vCPUs
☐ 15 GB memory, 8 vCPUs
☐ 70 GB memory, 36 vCPUs
☐ 145 GB memory, 72 vCPUs

Environment variables

Name	Value	Type	
		Plaintext	Remove

File systems

Identifier	ID	
	Q	Remove

Directory path - optional
Mount point
Mount options - optional

Figure 3.10: Environment compute and optional variable specifications

Environment variables

Name	Value	Type	
		Plaintext	Remove

File systems

Identifier	ID	
	Q	Remove

Directory path - optional
Mount point
Mount options - optional

Figure 3.11: Retaining default file system and parameter settings

Buildspec Configuration

Under the **Buildspec** section, select **Use a buildspec file**. AWS CodeBuild will inspect the root directory of the application repository for a file named `buildspec.yml` containing declarative build lifecycle phases.

Buildspec

Build specifications
☐ Insert build commands
 Store build commands as build project configuration

☒ Use a buildspec file
 Store build commands in a YAML-formatted buildspec file

Buildspec name - optional
 By default, CodeBuild looks for a file named buildspec.yml in the source code root directory. If your buildspec file uses a different name or location, enter its path from the source root here (for example, buildspec-two.yml or configuration/buildspec.yml).

Batch configuration
 You can run a group of builds as a single execution. Batch configuration is also available in advanced option when starting build.

☐ Define batch configuration - optional
 You can also define or override batch configuration when starting a build batch.

Figure 3.12: Choosing 'Use a buildspec file' (buildspec.yml)

Under **Logs**, check **CloudWatch logs** to stream real-time output to `/aws/codebuild/library-management-service-build-project`. Finally, click the orange **Continue to CodePipeline** button.

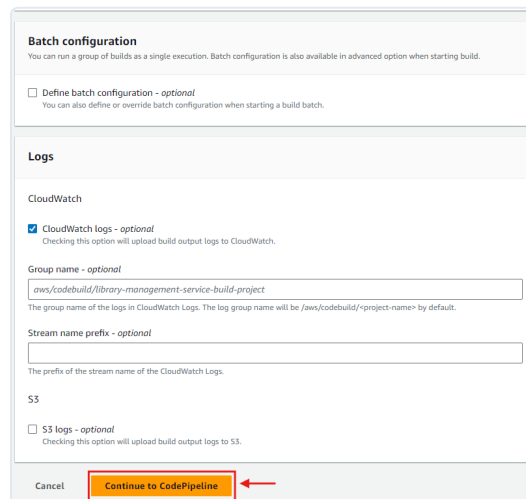


Figure 3.13: Enabling CloudWatch Logs and confirming project creation

Back in the CodePipeline build stage wizard, you will see a green confirmation: "Successfully created library-management-service-build-project in CodeBuild". Choose **Single build** as the build type and click **Next**.

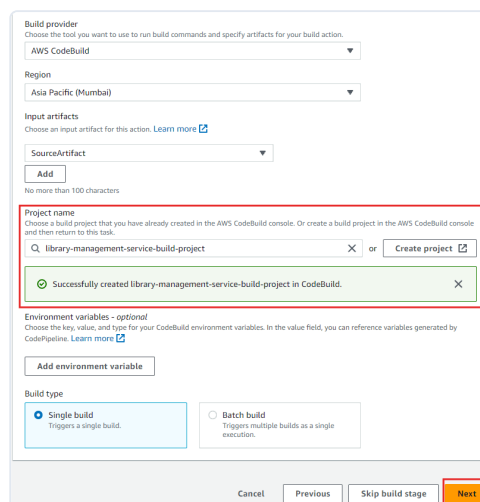


Figure 3.14: CodeBuild project verified and linked to CodePipeline build stage

Step 5 Add Deploy Stage (AWS Elastic Beanstalk)

Configure the production deployment target where the repackaged Spring Boot executable JAR will run:

- **Deploy provider:** Select **AWS Elastic Beanstalk**.
- **Region:** Select **Asia Pacific (Mumbai)**.
- **Input artifacts:** Select **BuildArtifact** (the output packaged during the CodeBuild phase).
- **Application name:** Select your Elastic Beanstalk application: **Library-management-service**.
- **Environment name:** Select target environment: **Library-management-service-env**.

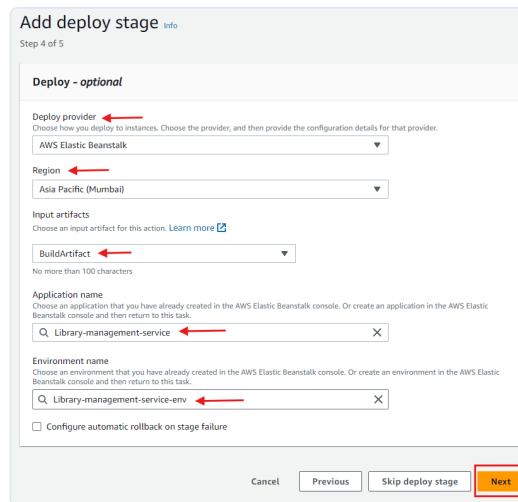


Figure 3.15: Configuring Elastic Beanstalk application and target environment

Step 6 Review & Pipeline Creation

Review the pipeline topology summary (Source → Build → Deploy) and click the orange **Create pipeline** button.

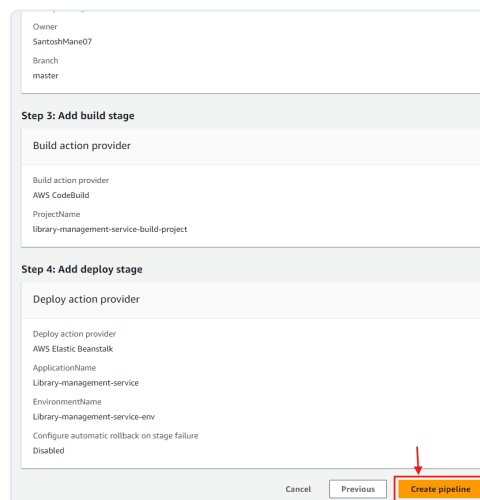


Figure 3.16: Reviewing action providers and initiating pipeline creation

4. Defining the Build Specification: buildspec.yml

Immediately after creation, the initial pipeline execution will fail because AWS CodeBuild cannot find a `buildspec.yml` file in the repository root. Create a file named `buildspec.yml` in your project root directory and commit it to GitHub.

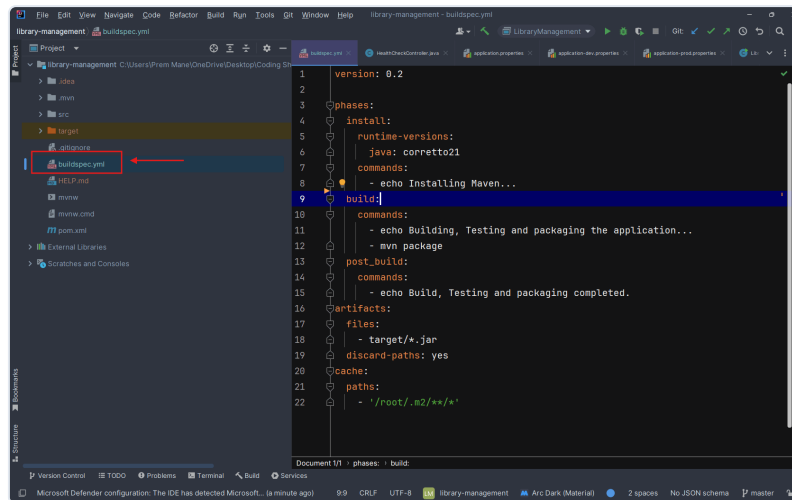


Figure 4.1: Project structure in IntelliJ IDEA showing `buildspec.yml` placed at the root level

BUILDSPEC.YML – SPRING BOOT BUILD & PACKAGE SPECIFICATION

```
version: 0.2
phases:
  install:
    runtime-versions:
      java: corretto21
    commands:
      - echo Installing Maven and setting up Corretto 21...
  build:
    commands:
      - echo Building, Testing and packaging the application...
      - mvn package
  post_build:
    commands:
      - echo Build, Testing and packaging completed.
artifacts:
  files:
    - target/*.jar
  discard-paths: yes
cache:
  paths:
    - '/root/.m2/**/*'
```

Anatomy of buildspec.yml:

- **install phase:** Declares runtime requirements (`java: corretto21`), allowing CodeBuild to inject Amazon Corretto 21 JDK dynamically.
- **build phase:** Executes `mvn package`, compiling source code, executing tests, and producing the Spring Boot JAR in `target/`.
- **post_build phase:** Confirms completion of the build and test life cycles.
- **artifacts:** Identifies generated files (`target/*.jar`) to forward as `BuildArtifact` to the deployment stage.
- **discard-paths: yes:** Strips preceding directory paths so only the flat JAR file is bundled, preventing nested folder resolution issues in Elastic Beanstalk.
- **cache (paths: '/root/.m2/**/*'):** Caches Maven repository dependencies between builds, drastically shortening build durations by avoiding repetitive dependency downloads.

Pro-Tip (Skipping Tests): If you need to quickly deploy without running unit tests during hotfixes, append the flag:

```
mvn package -DskipTests
```

5. Advanced Build Setup: Docker Hub & Testcontainers

When running integration tests backed by Testcontainers or requiring Docker inside CodeBuild, you must configure Docker Hub credentials and enable Privileged mode in AWS CodeBuild.

In the AWS Console, navigate to **CodeBuild** → **Build projects** → **library-management-service-build-project** → **Edit** → **Project configuration / Environment**:

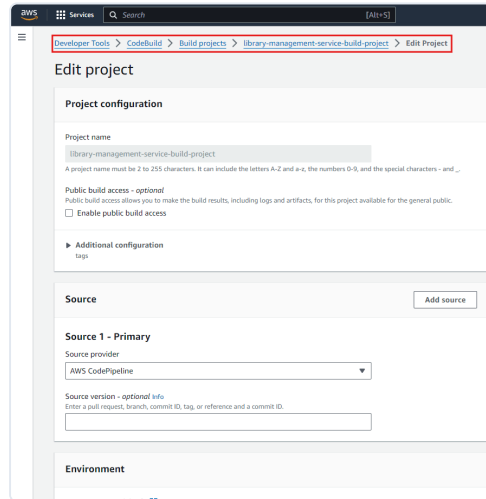


Figure 5.1: Navigating to Edit Project in AWS CodeBuild

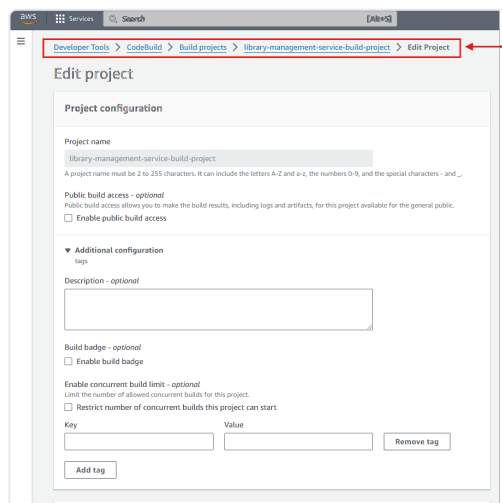


Figure 5.2: Project configuration and metadata overview

Step A: Enable Privileged Mode

Scroll down to **Additional configuration** within the Environment section. Check **Privileged**: "Enable this flag if you want to build Docker images or want your builds to get elevated privileges".

The screenshot shows the AWS CodeBuild console interface. Under the 'Additional configuration' section, the 'Privileged' checkbox is checked, indicating that the build is configured to build Docker images or require elevated privileges. Other visible settings include the image (aws/codebuild/amazonlinux2-x86_64-standard:5.0) and various timeout settings.

Figure 5.3: Enabling the Docker Privileged flag in CodeBuild

Step B: Configure Docker Credentials as Environment Variables

Under **Environment variables**, click **Add environment variable** and define:

- `DOCKER_USERNAME` → Your Docker Hub account username
- `DOCKER_PASSWORD` → Your Docker Hub personal access token or password

The screenshot shows the 'Environment variables' section of the AWS CodeBuild console. Two environment variables have been added: `DOCKER_USERNAME` with the value `yourdockerusername` and `DOCKER_PASSWORD` with the value `yourdockerpassword`. Both are set to 'Plaintext' type. The 'Add environment variable' button is visible at the bottom of the list.

Figure 5.4: Adding `DOCKER_USERNAME` and `DOCKER_PASSWORD` environment variables

Incorporate the login command inside a `pre_build` block in `buildspec.yml`:

PRE-BUILD PHASE DOCKER LOGIN

```
pre_build: commands: - echo Logging in to Docker Hub... - echo $DOCKER_PASSWORD | docker
login -u $DOCKER_USERNAME --password-stdin
```

Step C: Save and Update Project

Scroll to the bottom of the page, confirm the IAM service role modification permissions, and click **Update project**.

The screenshot shows the 'Update project' dialog in the AWS CodePipeline console. The 'Logs' section is expanded, showing options for 'CloudWatch logs - optional' (checked) and 'S3 logs - optional' (unchecked). The 'Service role permissions' section shows a selected service role 'arn:awsiam:156263551373:role/service-role/codebuild-library-management-' and a checked option 'Allow AWS CodeBuild to modify this service role so it can be used with this build project'. At the bottom, there are 'Cancel' and 'Update project' buttons, with a red arrow pointing to the 'Update project' button.

Figure 5.5: Saving configuration updates via 'Update project'

6. Pipeline Execution & Deployment Verification

With `buildspec.yml` committed and pushed to the `master` branch, CodePipeline detects the webhook event and kicks off the full CI/CD run automatically.

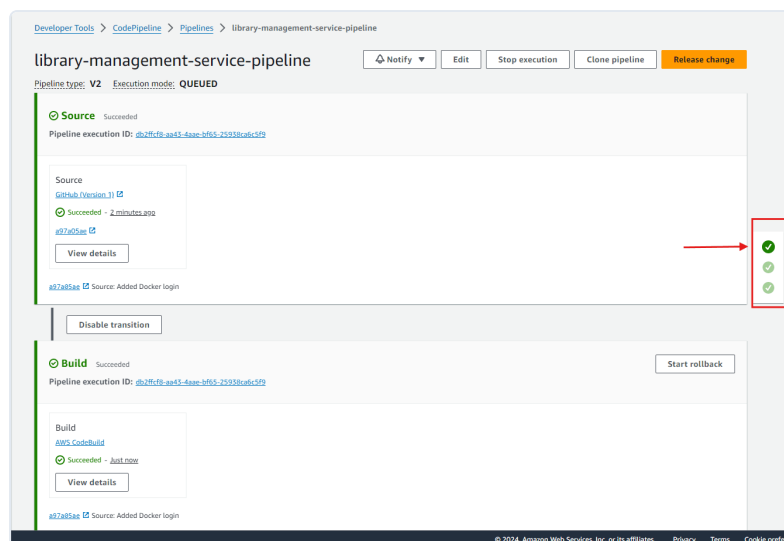


Figure 6.1: Pipeline execution in progress: Source and Build stages succeeded

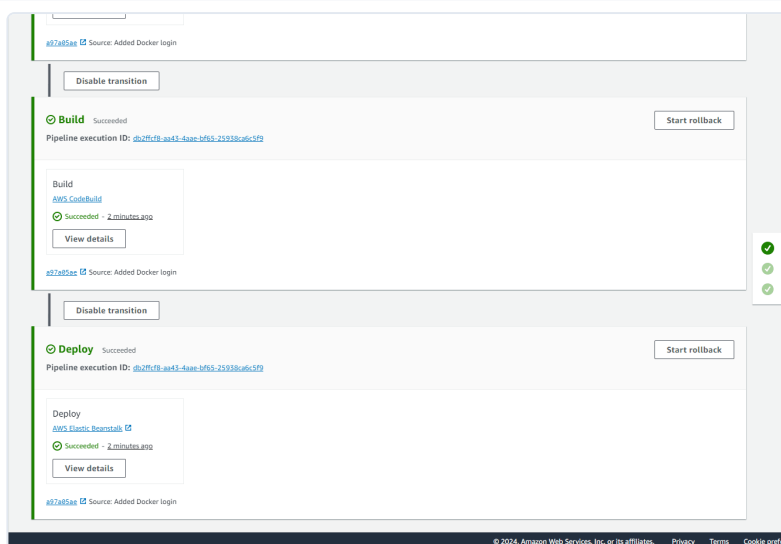


Figure 6.2: Complete end-to-end pipeline execution: Source, Build, and Deploy all green

Inspecting CodeBuild Execution Logs

Click **View details** under the Build stage to inspect execution logs in real time. The log stream confirms that all Maven compilation steps, unit tests, artifact archiving, and CloudWatch stream uploads succeeded without errors:

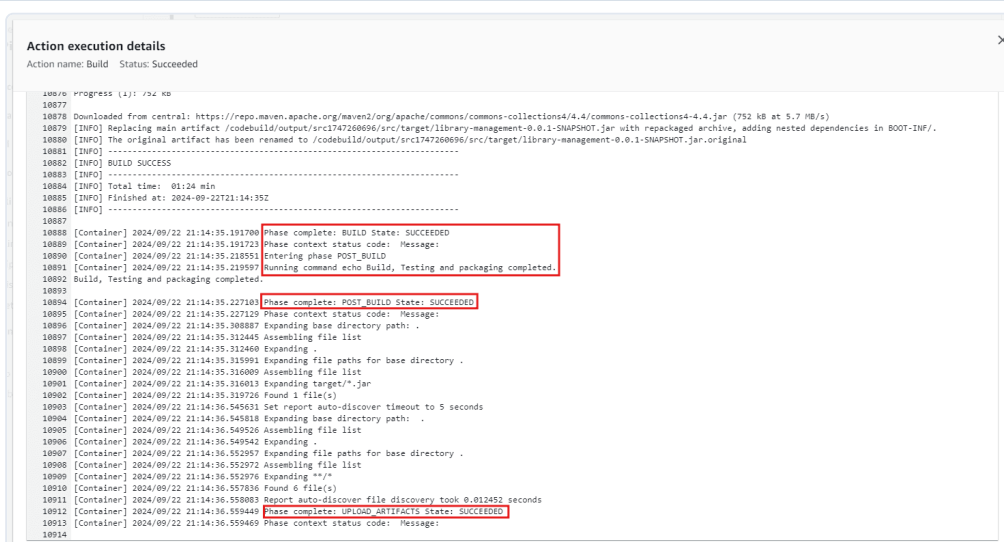
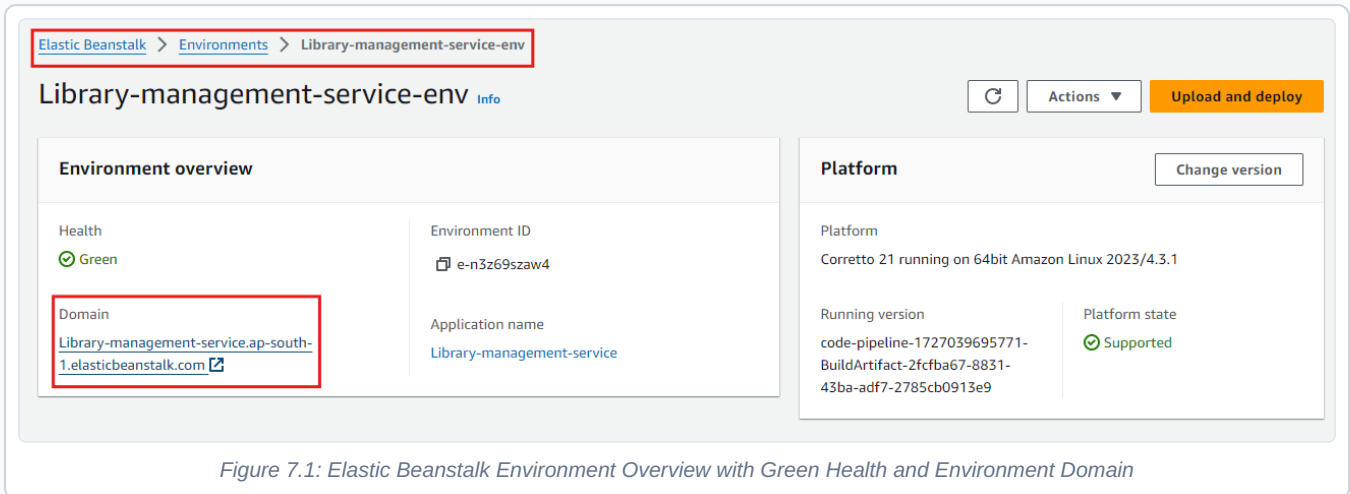


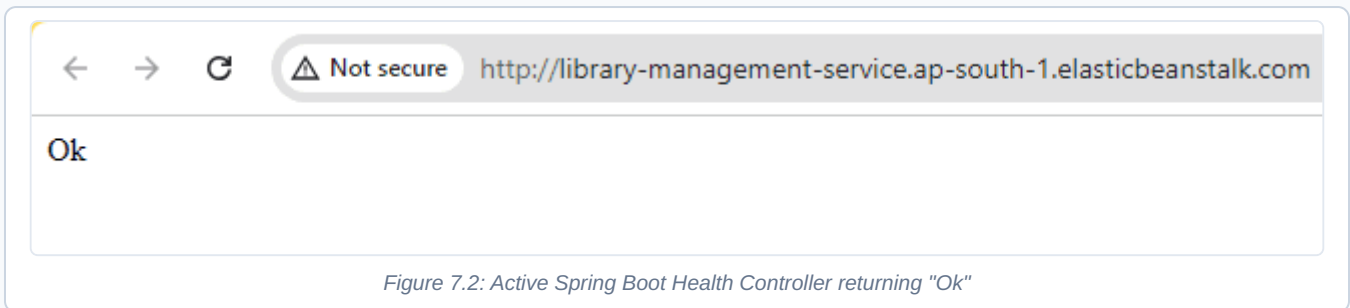
Figure 6.3: Action execution details confirming BUILD, POST_BUILD, and UPLOAD_ARTIFACTS SUCCEEDED

7. Production Health Check on AWS Elastic Beanstalk

Navigate to your **Elastic Beanstalk** environment dashboard (`Library-management-service-env`). Observe that the health state shows **✓ Green**, the platform is running Corretto 21 on 64bit Amazon Linux 2023, and the active version matches the generated build artifact ID.



Click on the application domain URL (`http://library-management-service.ap-south-1.elasticbeanstalk.com`) in your browser. The Spring Boot application responds with an HTTP 200 OK status from the health controller:



Deployment Complete: You have successfully built an automated, production-ready continuous integration and continuous deployment pipeline on AWS! Any subsequent push to your repository's master branch will automatically compile, test, build, and deploy your application across the globe in minutes.